

Student Depression Dataset

Importing Libraries

```
In [1]: # Basic libraries every data scientist uses
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier # step by step accuracy improve

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (accuracy_score, classification_report,
                             mean_absolute_error, r2_score)

import warnings
warnings.filterwarnings('ignore')
```

Data Preprocessing

◆ 1. Load Dataset

```
In [2]: data=pd.read_csv('Student Depression Dataset.csv')  
data
```

Out[2]:

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Diagnosis
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate
...	...	...	...	...	...	...	...	...	...	...	...	...
27896	140685	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhealthy
27897	140686	Male	27.0	Ludhiana	Student	2.0	0.0	9.40	3.0	0.0	Less than 5 hours	Healthy
27898	140689	Male	31.0	Faridabad	Student	3.0	0.0	6.61	4.0	0.0	5-6 hours	Unhealthy
27899	140690	Female	18.0	Ludhiana	Student	5.0	0.0	6.88	2.0	0.0	Less than 5 hours	Healthy
27900	140699	Male	27.0	Patna	Student	4.0	0.0	9.24	1.0	0.0	Less than 5 hours	Healthy

27901 rows × 18 columns



```
In [3]: data.head()
```

```
Out[3]:
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Ph
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate	
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate	M.



```
In [4]: data.tail()
```

Out[4]:

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits
27896	140685	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhealthy
27897	140686	Male	27.0	Ludhiana	Student	2.0	0.0	9.40	3.0	0.0	Less than 5 hours	Healthy
27898	140689	Male	31.0	Faridabad	Student	3.0	0.0	6.61	4.0	0.0	5-6 hours	Unhealthy
27899	140690	Female	18.0	Ludhiana	Student	5.0	0.0	6.88	2.0	0.0	Less than 5 hours	Healthy
27900	140699	Male	27.0	Patna	Student	4.0	0.0	9.24	1.0	0.0	Less than 5 hours	Healthy

In [5]: `data.describe().T`

Out[5]:

	count	mean	std	min	25%	50%	75%	max
id	27901.0	70442.149421	40641.175216	2.0	35039.00	70684.00	105818.00	140699.0
Age	27901.0	25.822300	4.905687	18.0	21.00	25.00	30.00	59.0
Academic Pressure	27901.0	3.141214	1.381465	0.0	2.00	3.00	4.00	5.0
Work Pressure	27901.0	0.000430	0.043992	0.0	0.00	0.00	0.00	5.0
CGPA	27901.0	7.656104	1.470707	0.0	6.29	7.77	8.92	10.0
Study Satisfaction	27901.0	2.943837	1.361148	0.0	2.00	3.00	4.00	5.0
Job Satisfaction	27901.0	0.000681	0.044394	0.0	0.00	0.00	0.00	4.0
Work/Study Hours	27901.0	7.156984	3.707642	0.0	4.00	8.00	10.00	12.0
Financial Stress	27898.0	3.139867	1.437347	1.0	2.00	3.00	4.00	5.0
Depression	27901.0	0.585499	0.492645	0.0	0.00	1.00	1.00	1.0

In [6]: `data.shape`

Out[6]: (27901, 18)

In [7]: `data.size`

Out[7]: 502218

In [8]: `data.dtypes`

```
Out[8]: id          int64
        Gender      object
        Age         float64
        City        object
        Profession   object
        Academic Pressure float64
        Work Pressure float64
        CGPA        float64
        Study Satisfaction float64
        Job Satisfaction float64
        Sleep Duration object
        Dietary Habits object
        Degree       object
        Have you ever had suicidal thoughts ? object
        Work/Study Hours float64
        Financial Stress float64
        Family History of Mental Illness object
        Depression   int64
        dtype: object
```

◆ 3. Check Missing Values

```
In [8]: data.isnull().sum()
```

```
Out[8]: id          0
        Gender      0
        Age         0
        City        0
        Profession   0
        Academic Pressure 0
        Work Pressure 0
        CGPA        0
        Study Satisfaction 0
        Job Satisfaction 0
        Sleep Duration 0
        Dietary Habits 0
        Degree       0
        Have you ever had suicidal thoughts ? 0
        Work/Study Hours 0
        Financial Stress 3
        Family History of Mental Illness 0
        Depression   0
        dtype: int64
```

◆ 4. Handle Missing Values

```
In [9]: data['Financial Stress'] = data['Financial Stress'].fillna(method='ffill')
```

◆ 7. Encoding (Convert Text → Numbers)

```
In [38]: le = LabelEncoder()
        data['Gender'] = le.fit_transform(data['Gender'])
```

```
In [39]: le1 = LabelEncoder()
        data['Profession'] = le1.fit_transform(data['Profession'])
```

```
In [40]: le2 = LabelEncoder()
        data['Academic Pressure'] = le2.fit_transform(data['Academic Pressure'])
```

```
In [41]: le3 = LabelEncoder()
        data['Sleep Duration'] = le3.fit_transform(data['Sleep Duration'])
```

```
In [42]: le4 = LabelEncoder()
data['Dietary Habits'] = le4.fit_transform(data['Dietary Habits'])
```

```
In [43]: le5 = LabelEncoder()
data['Degree'] = le5.fit_transform(data['Degree'])
```

```
In [44]: le6 = LabelEncoder()
data['Have you ever had suicidal thoughts ?'] = le6.fit_transform(data['Have you ever had suicidal thoughts ?'])
```

```
In [45]: le7 = LabelEncoder()
data['Family History of Mental Illness'] = le7.fit_transform(data['Family History of Mental Illness'])
```

```
In [28]: le8 = LabelEncoder()
data['City'] = le8.fit_transform(data['City'])
```

```
In [18]: data.dtypes
```

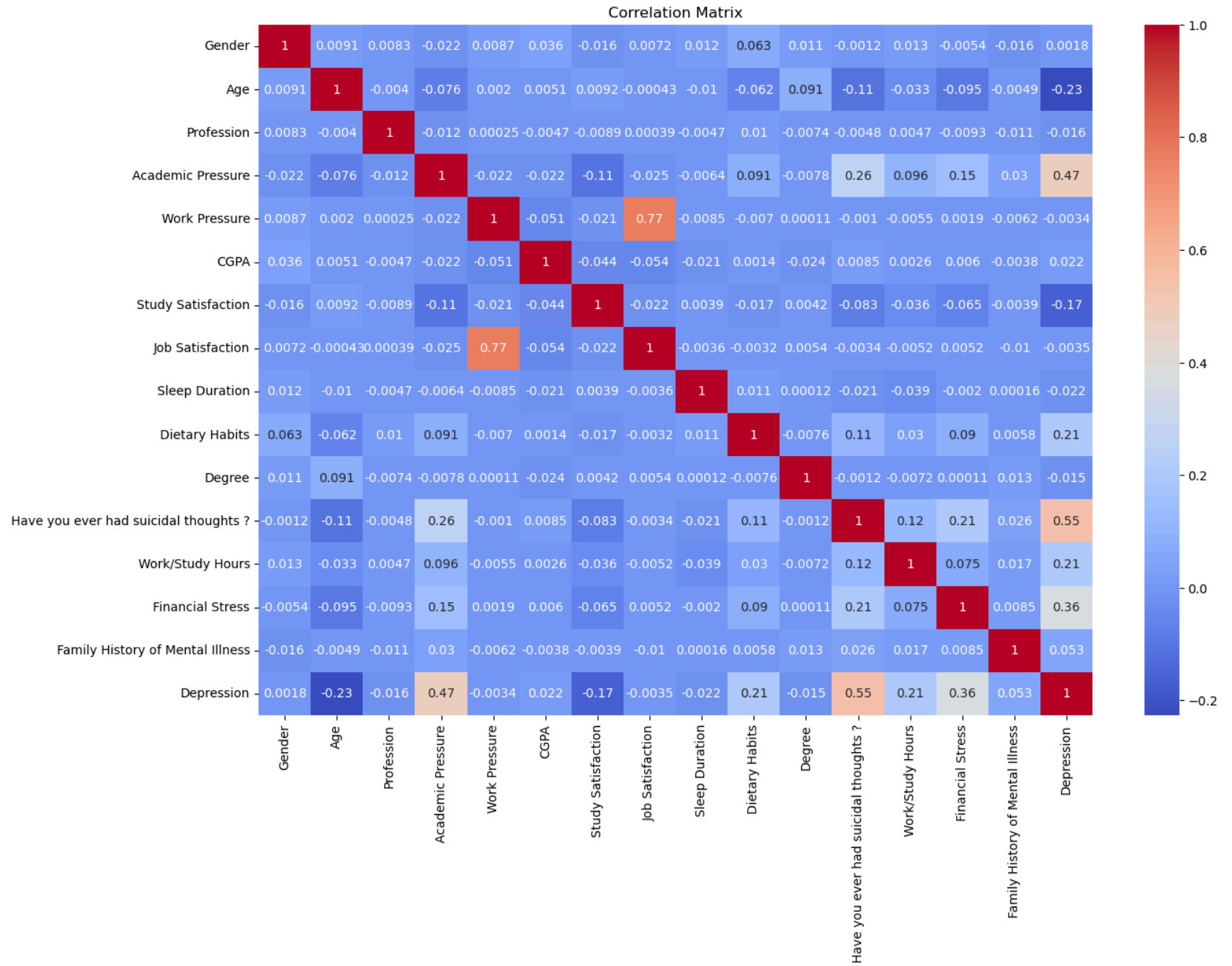
```
Out[18]: id                int64
Gender                int64
Age                  float64
City                 object
Profession           int64
Academic Pressure     int64
Work Pressure        float64
CGPA                 float64
Study Satisfaction   float64
Job Satisfaction     float64
Sleep Duration       int64
Dietary Habits       int64
Degree              int64
Have you ever had suicidal thoughts ? int64
Work/Study Hours     float64
Financial Stress     float64
Family History of Mental Illness      int64
Depression           int64
dtype: object
```

◆ 5. Remove Unnecessary Columns

```
In [19]: data = data.drop('id', axis=1)
```

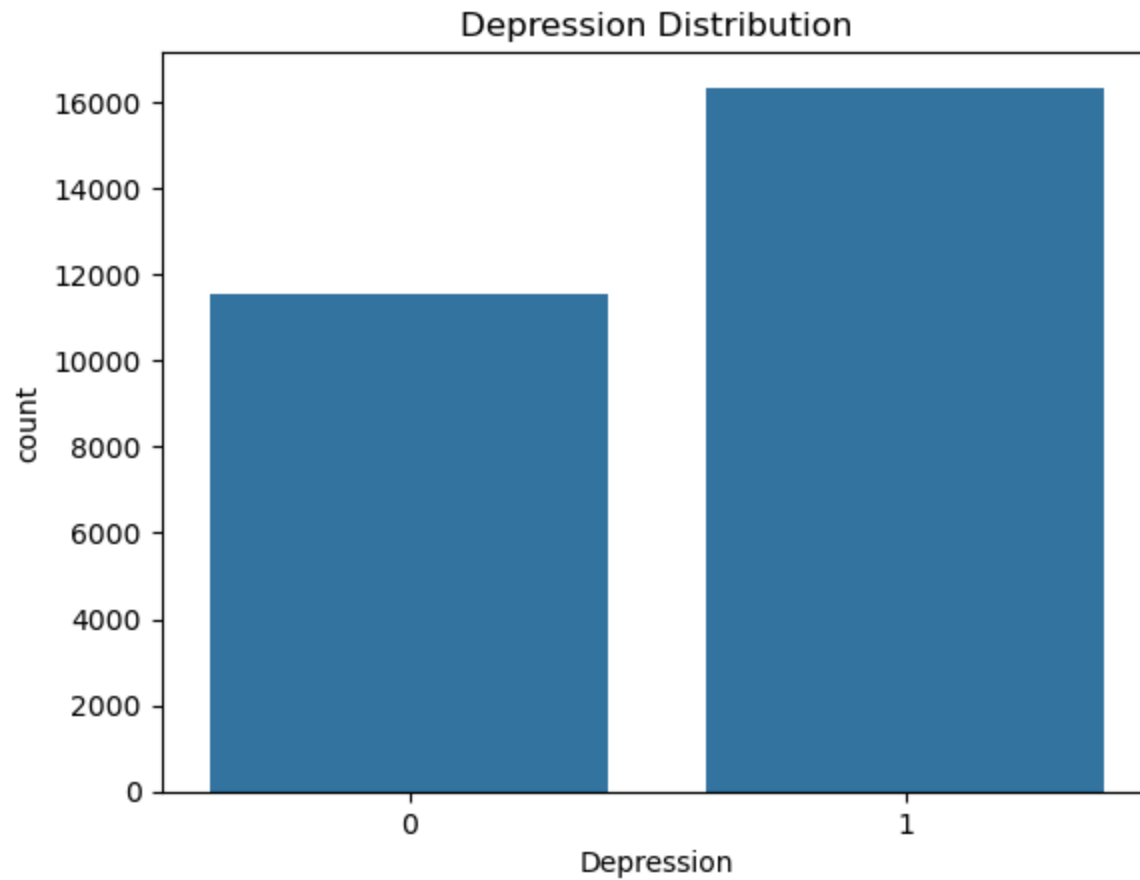
Correlation heatmap

```
In [20]: # Correlation heatmap
plt.figure(figsize=(15,10))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



◆ Exploratory Data Analysis (EDA)

```
In [21]: sns.countplot(x='Depression', data=data)  
plt.title("Depression Distribution")  
plt.show()
```



Split Data

```
In [22]: X = data.drop('Depression', axis=1)  
y = data['Depression']
```

```
In [23]: x_train, x_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)
```

```
In [34]: y_train.value_counts()
```

```
Out[34]: Depression
1      13068
0       9252
Name: count, dtype: int64
```

```
In [48]: from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
x_train_resampled, y_train_resampled = smote.fit_resample(x_train, y_train)

# Check class distribution after resampling
print(y_train_resampled.value_counts())
```

```
Depression
1      13068
0      13068
Name: count, dtype: int64
```

```
In [36]: scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

```
In [37]: plt.style.use('default')
```

Model Training, Evaluation, and Tuning

KNN

```
In [38]: knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(x_train,y_train)
y_testpred=knn.predict(x_test)
y_trainpred=knn.predict(x_train)
print('Test Score:',knn.score(x_test,y_test))
print('Train Score:',knn.score(x_train,y_train))
```

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test, y_testpred))
print('CR:', classification_report(y_test, y_testpred))
knnt = accuracy_score(y_test, y_testpred) * 100
knnr = accuracy_score(y_train, y_trainpred) * 100
print('Test Accuracy:', knnt)
print('Train Accuracy:', knnr)

```

Test Score: 0.8018276294570865

Train Score: 0.9070630547903276

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x00000216297EDD30>

CR:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.76	0.76	0.76	2313
---	------	------	------	------

1	0.83	0.83	0.83	3268
---	------	------	------	------

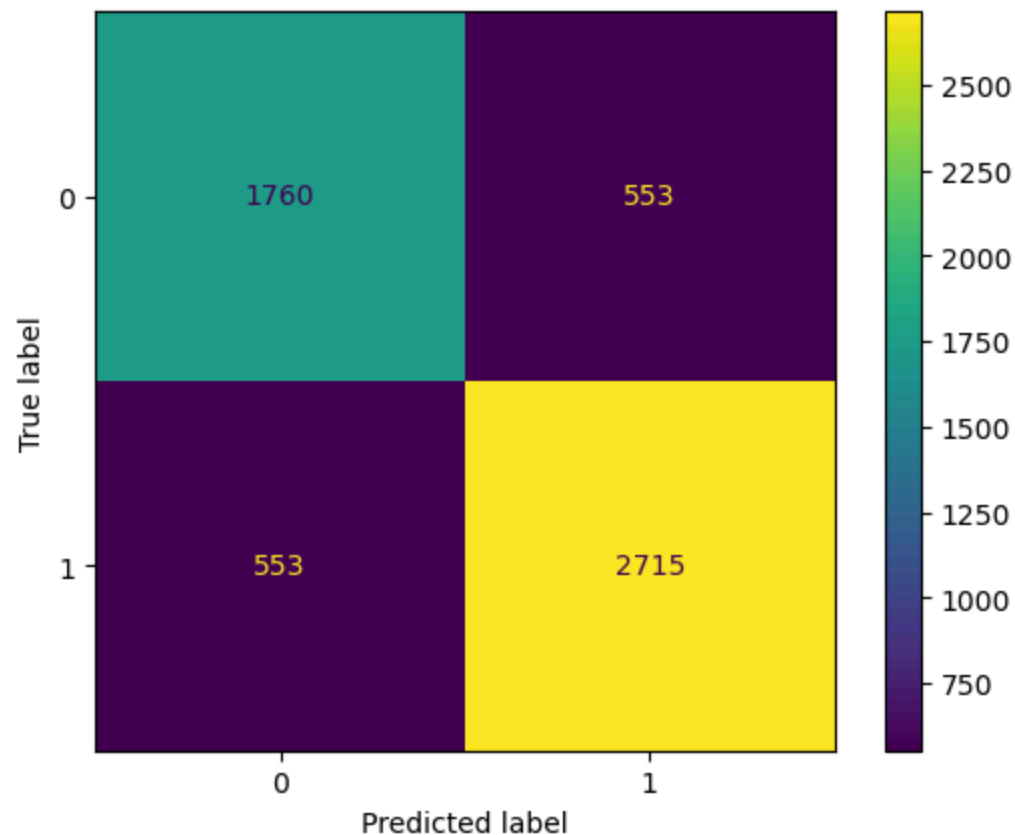
accuracy			0.80	5581
----------	--	--	------	------

macro avg	0.80	0.80	0.80	5581
-----------	------	------	------	------

weighted avg	0.80	0.80	0.80	5581
--------------	------	------	------	------

Test Accuracy: 80.18276294570865

Train Accuracy: 90.70630547903275



2 RANDOM FOREST CLASSIFIER

```
In [39]: rf=RandomForestClassifier(n_estimators=5,random_state=10)
rf.fit(x_train,y_train)
y_testpred1=rf.predict(x_test)
y_trainpred1=rf.predict(x_train)
print('Test Score:', rf.score(x_test,y_test))
print('Train Score:', rf.score(x_train,y_train))
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test,y_testpred1))
print('CR:',classification_report(y_test,y_testpred1))
rft=accuracy_score(y_test,y_testpred1)*100
rfr=accuracy_score(y_train,y_trainpred1)*100
```

```
print('Test Accuracy:',rft)
print('Train Accuracy:',rfr)
```

Test Score: 0.8175954130084214

Train Score: 0.980907560453015

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x000002162A301F90>

CR: precision recall f1-score support

0	0.78	0.78	0.78	2313
---	------	------	------	------

1	0.84	0.85	0.84	3268
---	------	------	------	------

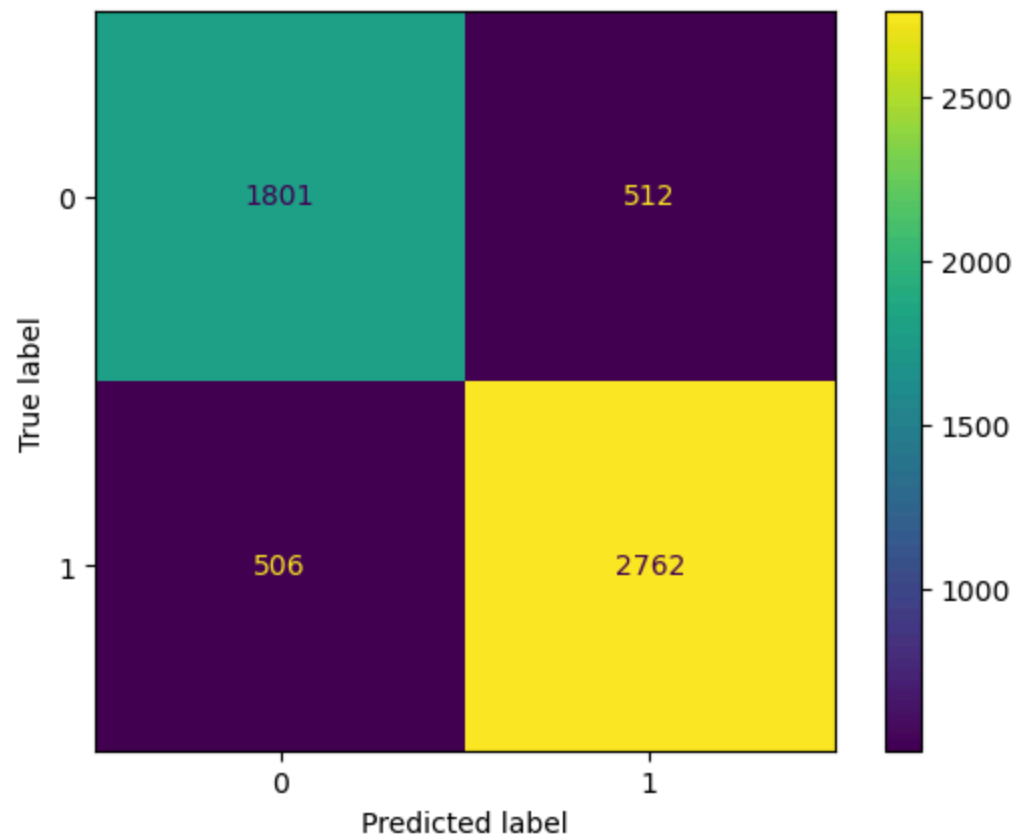
accuracy			0.82	5581
----------	--	--	------	------

macro avg	0.81	0.81	0.81	5581
-----------	------	------	------	------

weighted avg	0.82	0.82	0.82	5581
--------------	------	------	------	------

Test Accuracy: 81.75954130084214

Train Accuracy: 98.0907560453015



3 SUPPORT VECTOR MACHINE (SVM)

```
In [40]: sv=SVC(kernel='linear',random_state=42)
sv.fit(x_train,y_train)
y_testpred2=sv.predict(x_test)
y_trainpred2=sv.predict(x_train)
print('Test Score:',sv.score(x_test,y_test))
print('Train Score:',sv.score(x_train,y_train))
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test,y_testpred2))
print('CR:',classification_report(y_test,y_testpred2))
svt=accuracy_score(y_test,y_testpred2)*100
svr=accuracy_score(y_train,y_trainpred2)*100
```

```
print('Test Accuracy:',svt)
print('Train Accuracy:',svr)
```

Test Score: 0.8387385773158932

Train Score: 0.8597337006427915

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x0000021628A56990>

CR: precision recall f1-score support

0	0.81	0.80	0.80	2313
---	------	------	------	------

1	0.86	0.86	0.86	3268
---	------	------	------	------

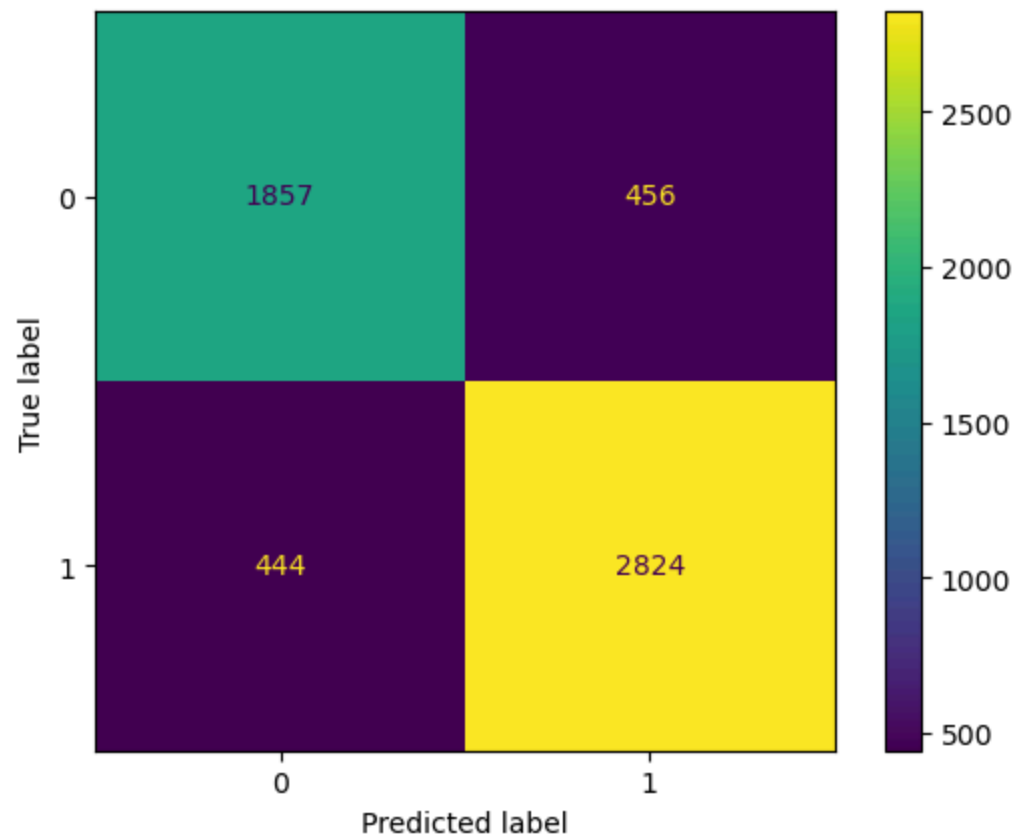
accuracy			0.84	5581
----------	--	--	------	------

macro avg	0.83	0.83	0.83	5581
-----------	------	------	------	------

weighted avg	0.84	0.84	0.84	5581
--------------	------	------	------	------

Test Accuracy: 83.87385773158933

Train Accuracy: 85.97337006427915



4) NAIVE BAYES CLASSIFIER

```
In [41]: gb=GaussianNB()
gb.fit(x_train,y_train)
y_testpred3=gb.predict(x_test)
y_trainpred3=gb.predict(x_train)
print('Test Score:',gb.score(x_test,y_test))
print('Train Score:',gb.score(x_train,y_train))
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test,y_testpred3))
print('CR:',classification_report(y_test,y_testpred3))
gbr=accuracy_score(y_test,y_testpred3)*100
gbr=accuracy_score(y_train,y_trainpred3)*100
```

```
print('Test Accuracy:',gbr)
print('Train Accuracy:',gbr)
```

Test Score: 0.5851997849847698

Train Score: 0.5001530456075911

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x00000216297BB820>

CR: precision recall f1-score support

0	0.00	0.00	0.00	2313
---	------	------	------	------

1	0.59	1.00	0.74	3268
---	------	------	------	------

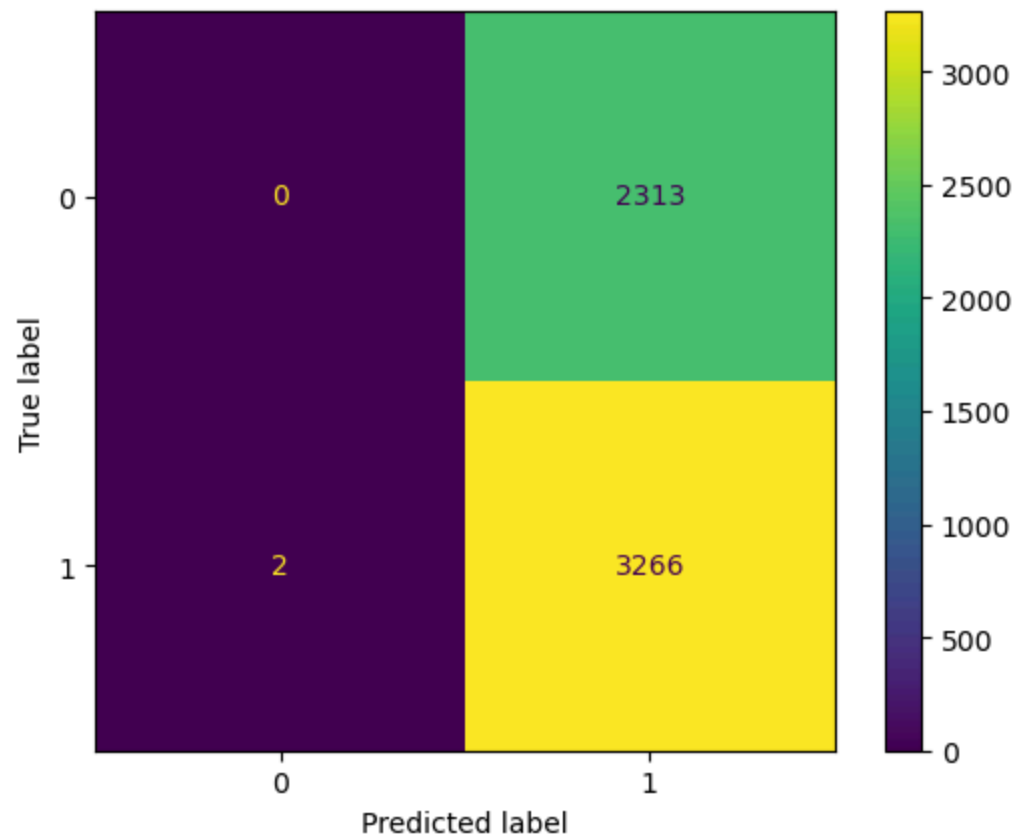
accuracy			0.59	5581
----------	--	--	------	------

macro avg	0.29	0.50	0.37	5581
-----------	------	------	------	------

weighted avg	0.34	0.59	0.43	5581
--------------	------	------	------	------

Test Accuracy: 58.51997849847697

Train Accuracy: 50.01530456075911



5) DECISION TREE CLASSIFIER

```
In [42]: dt=DecisionTreeClassifier(criterion='entropy')
dt.fit(x_train,y_train)
y_testpred4=dt.predict(x_test)
y_trainpred4=dt.predict(x_train)
print('Test Score:', dt.score(x_test,y_test))
print('Train Score:', dt.score(x_train,y_train))
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test,y_testpred4))
print('CR:',classification_report(y_test,y_testpred4))
dtt=accuracy_score(y_test,y_testpred4)*100
dtr=accuracy_score(y_train,y_trainpred4)*100
```

```
print('Test Accuracy:', dtt)  
print('Train Accuracy:', dtr)
```

Test Score: 0.7620498118616735

Train Score: 1.0

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x00000216297BB820>

CR: precision recall f1-score support

0	0.71	0.73	0.72	2313
---	------	------	------	------

1	0.80	0.79	0.79	3268
---	------	------	------	------

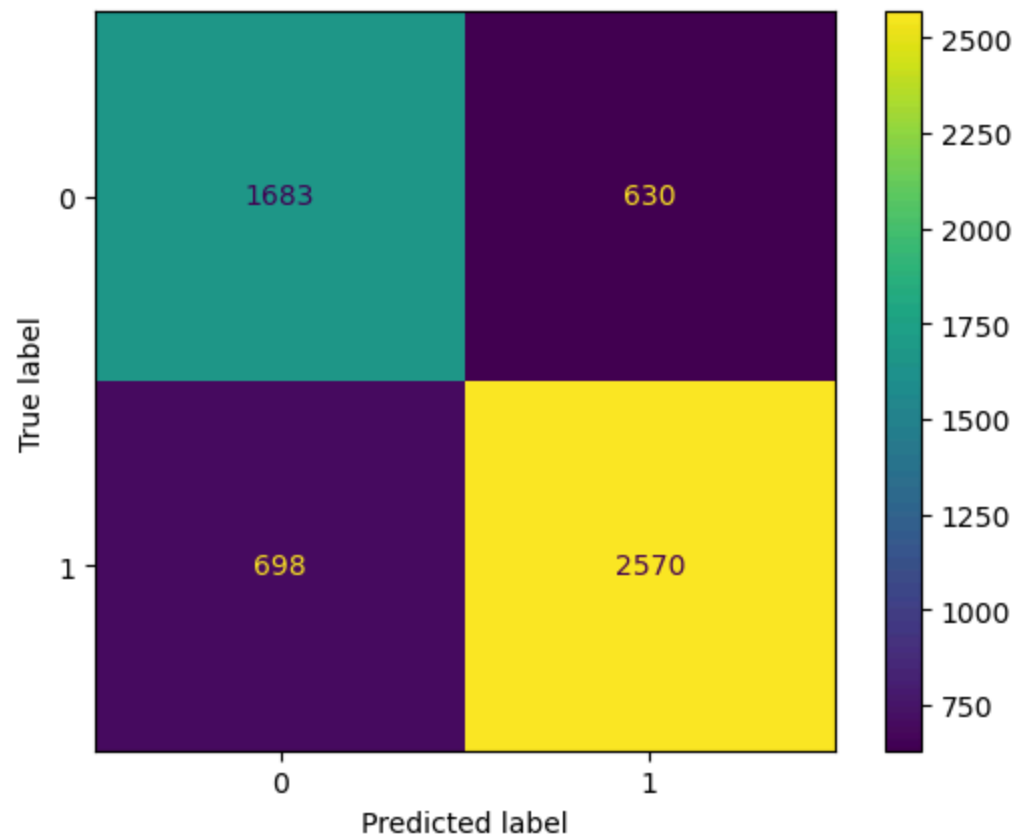
accuracy			0.76	5581
----------	--	--	------	------

macro avg	0.75	0.76	0.76	5581
-----------	------	------	------	------

weighted avg	0.76	0.76	0.76	5581
--------------	------	------	------	------

Test Accuracy: 76.20498118616736

Train Accuracy: 100.0



6) LOGISTIC REGRESSION

```
In [43]: lr=LogisticRegression()
lr.fit(x_train,y_train)
y_testpred5=lr.predict(x_test)
y_trainpred5=lr.predict(x_train)
print('Test Score:', lr.score(x_test,y_test))
print('Train Score:', lr.score(x_train,y_train))
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
print(ConfusionMatrixDisplay.from_predictions(y_test,y_testpred5))
print('CR:',classification_report(y_test,y_testpred5))
lrr=accuracy_score(y_test,y_testpred5)*100
lrr=accuracy_score(y_train,y_trainpred5)*100
```

```
print('Test Accuracy:',lrt)
print('Train Accuracy:',lrr)
```

Test Score: 0.8385593979573553

Train Score: 0.8605371900826446

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x000002162A35C050>

CR: precision recall f1-score support

0	0.80	0.81	0.81	2313
---	------	------	------	------

1	0.86	0.86	0.86	3268
---	------	------	------	------

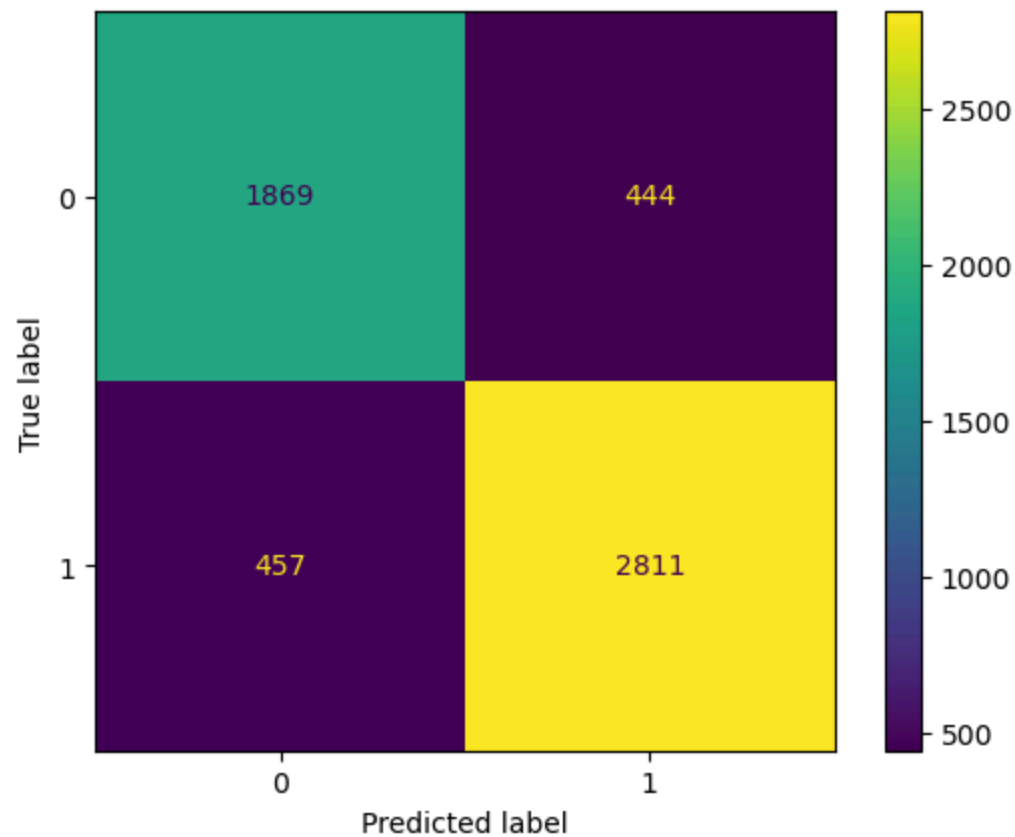
accuracy			0.84	5581
----------	--	--	------	------

macro avg	0.83	0.83	0.83	5581
-----------	------	------	------	------

weighted avg	0.84	0.84	0.84	5581
--------------	------	------	------	------

Test Accuracy: 83.85593979573554

Train Accuracy: 86.05371900826447



7 AdaBoost Classifier

```
In [44]: ab = AdaBoostClassifier()
ab.fit(x_train, y_train)

y_testpred_ab = ab.predict(x_test) #ws
y_trainpred_ab = ab.predict(x_train)

print('Test Score:', ab.score(x_test, y_test))
print('Train Score:', ab.score(x_train, y_train))

print(ConfusionMatrixDisplay.from_predictions(y_test, y_testpred_ab))

print('CR:', classification_report(y_test, y_testpred_ab))
```

```

abt = accuracy_score(y_test, y_testpred_ab) * 100
abr = accuracy_score(y_train, y_trainpred_ab) * 100
print('Test Accuracy:', abt)
print('Train Accuracy:', abr)

```

Test Score: 0.8462641103744849

Train Score: 0.8649755127027854

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x0000021629B20D10>

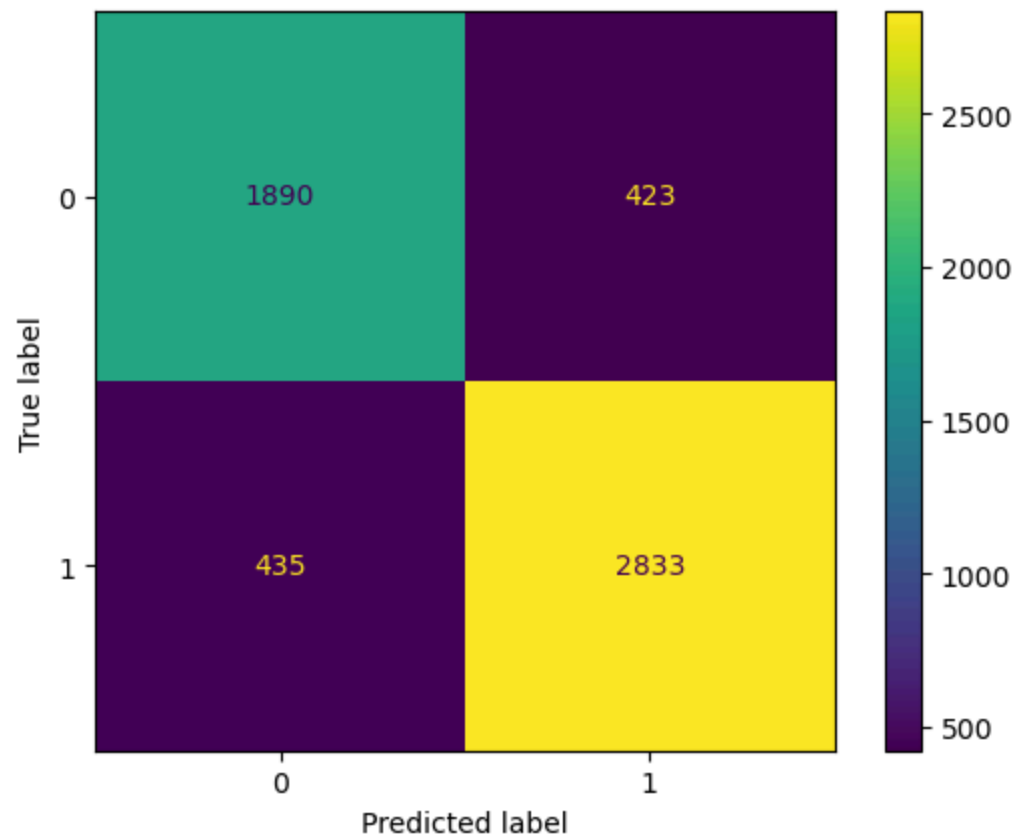
CR: precision recall f1-score support

0	0.81	0.82	0.82	2313
1	0.87	0.87	0.87	3268

accuracy			0.85	5581
macro avg	0.84	0.84	0.84	5581
weighted avg	0.85	0.85	0.85	5581

Test Accuracy: 84.62641103744849

Train Accuracy: 86.49755127027854



8 Gradient Boosting Classifier

```
In [45]: gbc = GradientBoostingClassifier()

gbc.fit(x_train, y_train)

y_testpred_gb = gbc.predict(x_test)
y_trainpred_gb = gbc.predict(x_train)

print('Test Score:', gbc.score(x_test, y_test))
print('Train Score:', gbc.score(x_train, y_train))

print(ConfusionMatrixDisplay.from_predictions(y_test, y_testpred_gb))
```

```

print('CR:', classification_report(y_test, y_testpred_gb))

gbct = accuracy_score(y_test, y_testpred_gb) * 100
gbcr = accuracy_score(y_train, y_trainpred_gb) * 100

print('Test Accuracy:', gbct)
print('Train Accuracy:', gbcr)

```

Test Score: 0.8484142626769396

Train Score: 0.8707912457912458

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x0000021629B22030>

CR:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.82	0.81	0.82	2313
---	------	------	------	------

1	0.87	0.88	0.87	3268
---	------	------	------	------

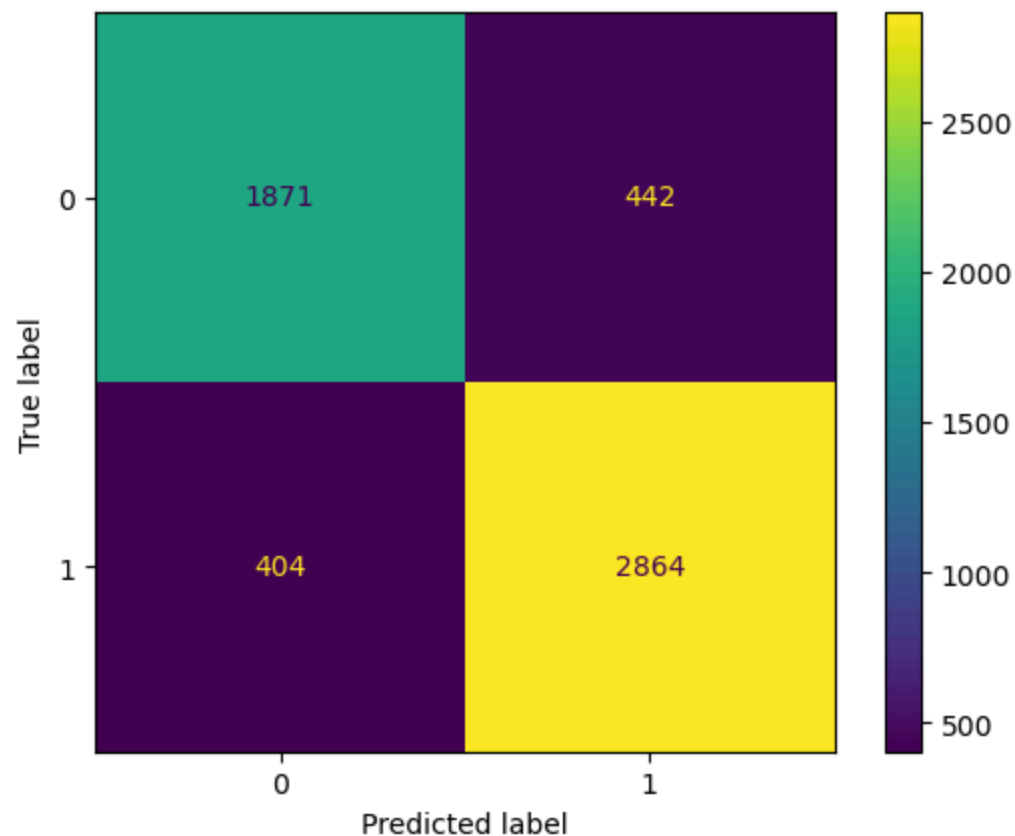
accuracy			0.85	5581
----------	--	--	------	------

macro avg	0.84	0.84	0.84	5581
-----------	------	------	------	------

weighted avg	0.85	0.85	0.85	5581
--------------	------	------	------	------

Test Accuracy: 84.84142626769396

Train Accuracy: 87.07912457912458



◆ Accuracy Comparison of Models

```
In [117... tb=pd.DataFrame()
tb['model']=pd.Series(['KNN','RANDOM FOREST','NAIVE BAYES','DECISION TREE','LOGISTIC REGRESSION','ADABOOST CLASSIFIER',
                      'GRADIENT BOOSTING CLASSIFIER'])
tb['test_accuracy']=pd.Series([knnt,rft,gbt,dtc,lrt,abt,gbct])
tb['train_accuracy']=pd.Series([knnt,rft,gbt,dtc,lrt,abt,gbct])
tb
#Gradient Boosting is an ensemble method that builds models step by step.
```

Out[117...

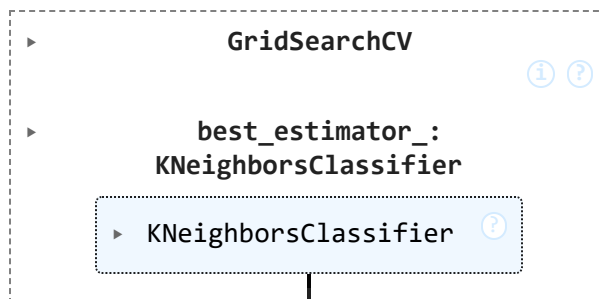
	model	test_accuracy	train_accuracy
0	KNN	80.182763	90.706305
1	RANDOM FOREST	81.759541	98.090756
2	NAIVE BAYES	58.519978	50.015305
3	DECISION TREE	76.599176	100.000000
4	LOGISTIC REGRESSION	83.855940	86.053719
5	ADABOOST CLASSIFIER	84.626411	86.497551
6	GRADIENT BOOSTING CLASSIFIER	84.841426	87.079125

HYPERPARAMETER TUNNING

```
In [46]: knn_param={
    'n_neighbors': range(3,7),
    'weights':['uniform','distance'],
    'metric':['euclidean','manhattan']
}

knn_grid=GridSearchCV(knn,knn_param,cv=5,scoring='accuracy',n_jobs=-1)
knn_grid.fit(x_train,y_train)
👉 In short: Hyperparameter tuning is used to improve model performance by finding the best settings for the algorithm
```

Out[46]:

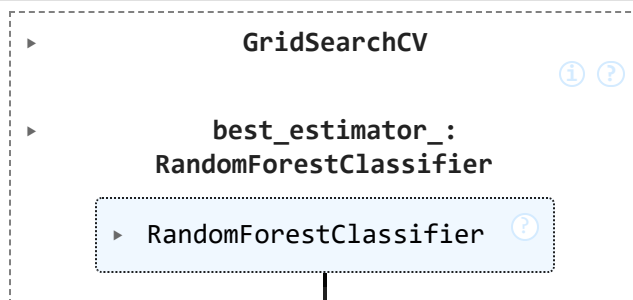


```
In [48]: # RANDOM FOREST

rf_params={'n_estimators':[50,100],'max_depth':[2,4]}
```

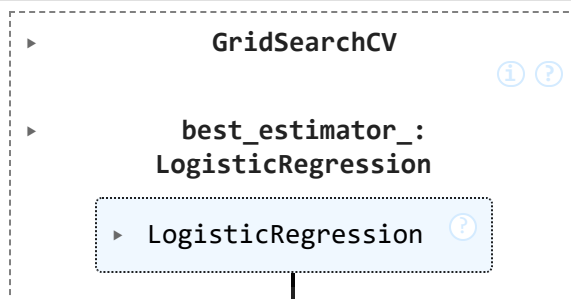
```
rf_grid=GridSearchCV(RandomForestClassifier(random_state=42),rf_params,cv=5)
rf_grid.fit(x_train,y_train)
```

Out[48]:



```
In [49]: # LOGISTIC REGRESSION
lr_params={'C':[0.1,1,10]}
lr_grid=GridSearchCV(LogisticRegression(),lr_params,cv=5)
lr_grid.fit(x_train,y_train)
```

Out[49]:

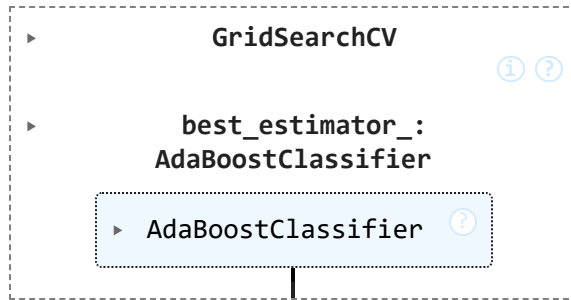


```
In [50]: # ADABOOST CLASSIFIER

ab_params = {
    'n_estimators': [25, 100],
    'learning_rate': [0.01, 0.1]
}

ab_grid = GridSearchCV(AdaBoostClassifier(random_state=42),ab_params,cv=5)
ab_grid.fit(x_train, y_train)
```

Out[50]:

In [51]: *# GRADIENT BOOSTING CLASSIFIER*

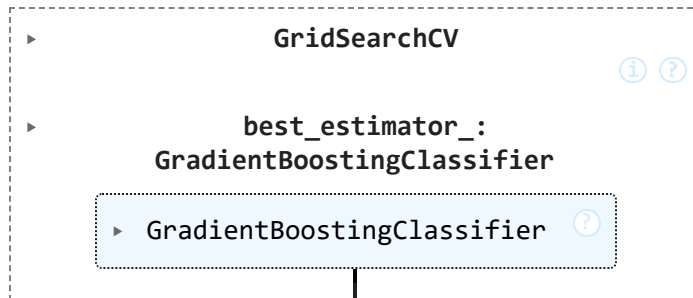
```

gb_params = {
    'n_estimators': [25, 100],
    'learning_rate': [ 0.1],
    'max_depth': [1,4]
}

gb_grid = GridSearchCV(GradientBoostingClassifier(random_state=42),gb_params,cv=5)
gb_grid.fit(x_train, y_train)

```

Out[51]:



```

In [52]: models = {
    'KNN': knn_grid,
    'Random Forest': rf_grid,
    'Naive Bayes': gb_grid,
    'Logistic Regression': lr_grid,
    'AdaBoost Classifier': ab_grid,
    'Gradient Boosting Classifier': gb_grid
}

for name, grid in models.items():

```

```

best_model = grid.best_estimator_
y_pred = best_model.predict(x_test)
acc = accuracy_score(y_test, y_pred)

print(f"{name}:")
print(f" Best Params: {grid.best_params_}")
print(f" Accuracy: {acc:.4f}\n")
# evaluate and compare the best-tuned versions of each classifier after hyperparameter tuning.

```

KNN:

Best Params: {'metric': 'manhattan', 'n_neighbors': 6, 'weights': 'distance'}
 Accuracy: 0.8235

Random Forest:

Best Params: {'max_depth': 4, 'n_estimators': 100}
 Accuracy: 0.8251

Naive Bayes:

Best Params: {'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 100}
 Accuracy: 0.8457

Logistic Regression:

Best Params: {'C': 0.1}
 Accuracy: 0.8382

AdaBoost Classifier:

Best Params: {'learning_rate': 0.1, 'n_estimators': 100}
 Accuracy: 0.8339

Gradient Boosting Classifier:

Best Params: {'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 100}
 Accuracy: 0.8457

```

In [53]: # Access the best model
best_gb_model = gb_grid.best_estimator_
y_pred = best_gb_model.predict(x_test)

```

```

In [54]: print(best_gb_model)

```

GradientBoostingClassifier(max_depth=4, random_state=42)

```
In [56]: import joblib

joblib.dump(best_gb_model, 'Depression_gb_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(le, 'le_gender.pkl')
joblib.dump(le1, 'le_profession.pkl')
joblib.dump(le2, 'le_academic_pressure.pkl')
joblib.dump(le3, 'le_sleep_duration.pkl')
joblib.dump(le4, 'le_dietary_habits.pkl')
joblib.dump(le5, 'le_degree.pkl')
joblib.dump(le6, 'le_suicidal_thoughts.pkl')
joblib.dump(le7, 'le_family_history.pkl')
joblib.dump(le8, 'le_city.pkl')
```

Out[56]: ['le_city.pkl']

In []: